PROJECT AND TEAM INFORMATION

Project Title

Mini Git: A Simple Version Control System Built from Scratch in C

Student / Team Information

Team Name: **Team ID:**	**CODE RUSHERS** DSCPP-III-2026-T358	
Team Member 1 (Team Lead) Student ID	Singh, Jasmeet 2510370305 2510370305@geu.ac.in	
Team Member 2 Student ID	Devraj Raushan 260211526 260211526@geu.ac.in	

Faculty: Prof. Dr. Deepak Arsani
Email: deepakarsani.cse@geu.ac.in

PROPOSAL DESCRIPTION

Motivation

Almost every software project uses version control. It is a tool that saves every change you make to your files. If something goes wrong, you can go back to an older version.

Most students use Git, but they only remember the commands, like commit, branch and merge. They do not know what happens inside. So when something breaks, such as a merge conflict or a lost commit, they do not know how to fix it.

This is important because companies want you to know Git for internships and jobs. The ideas inside Git are also part of our Data Structures course. These ideas are hashing, linked lists, graphs and dynamic programming. Real Git is very big and hard to read. We want a small project that you can read fully, but it must still really work.

By building Mini Git ourselves, we learn how Git saves files. We also learn how it keeps history, compares two files (diff) and joins two branches (merge). Beginners and teachers can read it, run it and extend it.

State of the Art / Current solution

There are two kinds of version control tools. In centralised tools like CVS and Subversion, one server keeps all the history. In distributed tools like Git and Mercurial, every user has a full copy.

Git saves a full snapshot of your files each time. A snapshot is like a photo of all files. It does not save only the small changes. Git gives every saved item a name made from its content. This name is called a hash. To find what changed between two files, these tools use the longest common subsequence (LCS) idea. The Myers algorithm is the standard fast way to do it.

Good learning material already exists, such as the Pro Git book, the Git internals documentation and "Git from the bottom up". But they explain the big, real Git. Real Git uses SHA-1, zlib compression, nested trees and packfiles. These hide the main idea from a beginner.

Our project does not try to beat them. It is a tiny C program that is easy to read. It shows the data structures behind each command. It also explains how fast each command is.

Project Goals and Milestones

Goals. (1) Build a Git-like system with the commands init, add, commit, status and log. (2) Add branches and checkout using a HEAD pointer. HEAD tells us which commit we are on. (3) Build a diff feature using the LCS or Myers algorithm. (4) Add three-way merge and mark conflicts clearly. (5) Use hashing, trees, graphs and dynamic programming together in one project. The system is for one user and works on one computer. It handles text files only.

Milestones (eight weeks, three phases):

- **M1 (Phase I):** write the proposal, list the requirements and design how objects are stored.
- **M2 (Phase I):** build the main commands init, add, commit, status, log and checkout in C.
- **M3 (Phase II):** add branches with HEAD and checkout of a branch.
- **M4 (Phase II):** build the diff engine using LCS.
- **M5 (Phase II):** build the commit graph, find the common parent (merge base) and do three-way merge.
- **M6 (Phase III):** use SHA-1 and zlib, move to C++ classes, draw the commit graph with Graphviz and write the final report.

Project Approach

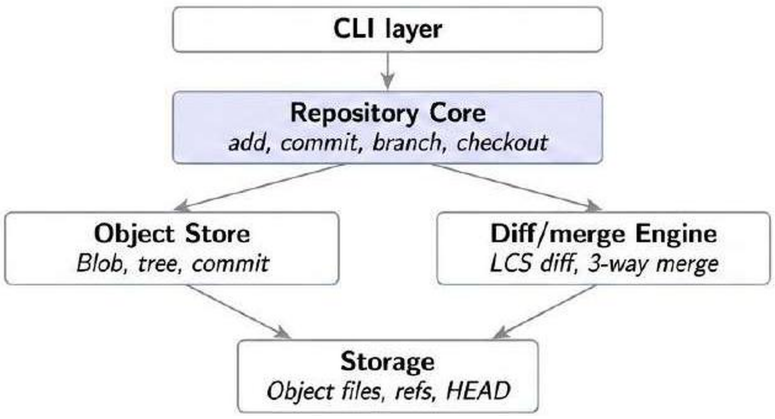
The project has five steps. First, understand the requirements. Then design the system, build it milestone by milestone, test it and check the results. The first version is written in C (about 300 lines, in the file minigit.c). It is built with gcc and runs on Linux and Windows without any extra library. In Phase III, we move to C++ classes (Blob, Tree, Commit, Branch and Repository).

Design. The system has four layers: CLI (the commands the user types), Repository Core, Diff/Merge Engine and Object Store. Each file is saved in .minigit/objects with a name made from its content (its hash), so the same content is saved only once. Each commit is a small text file. It stores the parent id, time, message and a list of (hash, path). The staging area (index) is a plain text file, just like in Git.

Techniques. 64-bit FNV-1a hashing (SHA-1 later), arrays and structs for Entry, Tree and Commit, a linked list of commits for log, BFS/DFS to find the merge base, and dynamic programming for the LCS diff.

Tools. gcc, OpenSSL and zlib (Phase III), Graphviz to draw graphs, Catch2 for C++ unit tests, and manual command-line tests for the first version.

System Architecture (High Level Diagram)



.minigit/ has: HEAD (id of the current commit), index (staging area), objects/ (file contents saved by hash) and commits/ (one text file per commit).

Project Outcome / Deliverables

- A working Mini Git in C with six commands: init, add, commit, status, log and checkout (Phase I, first version).
- Branches with HEAD, LCS-based diff and three-way merge that marks conflicts (Phase II).
- SHA-1 hashing, zlib compression, C++ classes and a Graphviz picture of the commit graph (Phase III).
- A study of how fast every command is (time complexity) and of hash collisions.
- Test cases (manual now, Catch2 later), a research paper in IEEE format, the final report and a presentation.

Assumptions

The system is for one user and works on one computer, so there are no remote repositories. It works on text files only. It tracks at most 256 files, with paths up to 512 characters, kept as one flat list without folders inside folders. Renamed files, binary diffs and file permissions are not handled. In the first version, we use FNV-1a as the hash because it is fast and easy to explain. We replace it with SHA-1 later.

References

[1] S. Chacon and B. Straub, *Pro Git*, 2nd ed., Apress, 2014. `https://git-scm.com/book/en/v2`

[2] "Git internals: Plumbing and porcelain," in *Pro Git*, ch. 10.

[3] "Git documentation." `https://git-scm.com/docs`

[4] J. Wiegley, "Git from the bottom up."

[5] E. W. Myers, "An O(ND) difference algorithm and its variations," *Algorithmica*, vol. 1, pp. 251–266, 1986.

[6] G. Fowler, L. C. Noll and K.-P. Vo, "FNV hash (Fowler–Noll–Vo)."

[7] T. H. Cormen et al., *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.